# Exercise 1: Structured NLP with JSON and Markdown Templates

*Module: Introduction to Prompt Engineering — Standard NLP Tasks*

**Purpose**

You will use a raw, messy dataset to perform two common NLP tasks: summarization and information extraction. The key skill is not only getting an answer, but controlling the format using structured JSON and Markdown templates.

## Student Exercise Overview

| Item | Details |
|---|---|
| Tools | ChatGPT, Mistral, Claude, Gemini, or another approved LLM |
| Main output | A Markdown summary report and a JSON extraction file |
| Core skill | Prompting the model to produce structured, verifiable output |

## Learning Objectives

- Write clear prompts for summarization and extraction tasks.
- Use JSON templates to force consistent extraction fields across multiple raw records.
- Use Markdown templates to produce readable summaries with headings, tables, and action items.
- Reduce hallucination by requiring evidence quotes, null values for missing information, and validation checks.
- Compare a first prompt with an improved prompt and explain what changed.

## Scenario

**You are helping the Student Services Office analyze feedback from a pilot campus shuttle tracking app.** The office collected raw comments from app reviews, help-desk emails, chat transcripts, and survey responses. The data are messy: some comments include multiple issues, some are vague, and some contain both positive and negative feedback. Your job is to use prompt engineering to convert this raw text into structured outputs that a manager can read and act on.

## Provided Raw Dataset

**Important rule**

Use only the information in the dataset below. Do not invent names, dates, causes, technical explanations, or solutions. If information is missing, use null in JSON and state the uncertainty in Markdown.

| Record | Raw note |
|---|---|
| **R01**<br>App review<br>2026-04-12 | "The bus arrival time is usually okay in the morning, but after 6 pm it is off by 15–20 minutes. I missed my lab section twice because the app said the shuttle was 3 minutes away." |
| **R02**<br>Help-desk email<br>2026-04-13 | "I was charged twice for the same monthly student shuttle pass. One charge says pending and one posted. Please refund the duplicate charge. I attached a screenshot, but I do not know which transaction ID you need." |
| **R03**<br>Survey response<br>2026-04-14 | "The app is useful, but the color contrast is hard to read outside. The green text on gray background is almost invisible in sunlight." |
| **R04**<br>Chat transcript<br>2026-04-15 | "Driver was friendly and helped me with my bag. The tracker froze at 7:42 for about ten minutes, then jumped to arrived. No crash message, just frozen." |
| **R05**<br>App review<br>2026-04-15 | "I like the notification when the shuttle is nearby. Can you add an option to notify me 10 minutes before arrival instead of only 2 minutes?" |
| **R06**<br>Help-desk email<br>2026-04-16 | "The wheelchair-accessible shuttle icon is not reliable. It showed accessible at North Gate, but the shuttle that arrived did not have a working ramp. I had to wait for the next one." |
| **R07**<br>Survey response<br>2026-04-16 | "It asks for location permission all the time. I understand location is needed while using the app, but why does it ask in the background? Please explain what data are stored." |
| **R08**<br>Chat transcript<br>2026-04-17 | "My route disappeared after I changed my phone language to Spanish. When I switched back to English, the route came back. Maybe the route names are not translated?" |
| **R09**<br>App review<br>2026-04-18 | "The shuttle is late at night near the dorms. The app still says on time, but students are waiting outside for a long time. This feels unsafe after dark." |
| **R10**<br>Survey response<br>2026-04-18 | "Overall much better than the old PDF schedule. I can see live buses and stop names. The only confusing part is that 'Arriving' sometimes means at the previous stop, not my stop." |
| **R11**<br>Help-desk email<br>2026-04-19 | "I bought a visitor day pass for my parent, but the QR code never loaded. We tried Wi-Fi and cellular. The payment went through, but the app just showed a spinning circle." |
| **R12**<br>App review<br>2026-04-20 | "Please add a simple text message option. Some students do not want to install another app just to know if the shuttle is delayed." |

## Your Task

Complete the following parts. You may run more than one prompt, but your final submission must include the prompts you used and the final outputs.

## Part 1 — Design the Prompt

- Write one prompt for summarization and one prompt for extraction. Each prompt should include: role, task, input data, output format, and rules.
- Your prompt must explicitly tell the model not to invent missing information.
- Your prompt must require evidence from the raw notes.

## Part 2 — Summarization Using Markdown

- Use the raw dataset to create a manager-facing summary in the required Markdown format.
- The summary should identify major themes, user impact, positive feedback, risks, and recommended next actions.
- Use record IDs as evidence, for example: R01, R06, R09.

## Part 3 — Extraction Using JSON

- Extract one structured JSON object for each raw record.
- Use only the controlled vocabulary given in the JSON template unless the field allows free text.
- Use null when the record does not contain enough information.

## Part 4 — Validate and Improve

- Check whether the JSON is valid and parseable.
- Check whether every extracted claim is supported by a raw note.
- Revise your prompt if the first output is vague, inconsistent, or not valid JSON.

## Part 5 — Reflection

- Write 150–200 words explaining how templates changed the quality of the model output.
- Mention one failure or limitation you observed and how you tried to fix it.

## Recommended Prompt Structure

```
Role: You are a careful NLP analyst.
Task: Analyze the provided raw dataset and perform [summarization / extraction].
Context: The dataset contains student feedback about a campus shuttle app.
Input: [Paste the raw dataset here.]
Output format: Follow the exact [Markdown / JSON] template below.
Rules:
- Use only information from the dataset.
- Do not invent missing details.
- Use record IDs as evidence.
- If a field is unknown, write null.
- Keep wording concise and manager-facing.
```

## Required Markdown Summary Template

```
# Campus Shuttle App Feedback Summary

## 1. One-Sentence Overview
[Write one sentence summarizing the overall feedback.]

## 2. Top Themes
| Theme | Evidence IDs | User Impact | Recommended Action |
|---|---|---|---|
| [Theme 1] | [R__ , R__] | [Impact] | [Action] |
| [Theme 2] | [R__ , R__] | [Impact] | [Action] |
| [Theme 3] | [R__ , R__] | [Impact] | [Action] |

## 3. Positive Feedback
- [Positive point with record ID]
- [Positive point with record ID]

## 4. High-Priority Risks
- [Risk, why it matters, evidence IDs]

## 5. Missing or Ambiguous Information
- [What information is missing?]

## 6. Recommended Next Steps
1. [Next step]
2. [Next step]
3. [Next step]
```

## Required JSON Extraction Template

Use this exact structure. The final JSON must be valid: no comments, no trailing commas, and double quotes around all keys and string values.

```
{
  "records": [
    {
      "id": "R01",
      "date": "YYYY-MM-DD or null",
      "source": "App review | Help-desk email | Survey response | Chat transcript",
      "issue_category": "timing_accuracy | payment_refund | accessibility | app_bug | safety |
privacy | feature_request | positive_feedback | unclear",
      "sentiment": "positive | mixed | negative | unclear",
      "urgency": "low | medium | high",
      "user_need": "short phrase describing what the user needs",
      "requested_action": "short phrase or null",
      "evidence_quote": "short quote copied from the raw note",
      "needs_follow_up": true,
      "missing_information": ["item 1", "item 2"]
    }
  ],
  "dataset_level_insights": {
    "most_common_issue_categories": ["category 1", "category 2", "category 3"],
    "highest_priority_record_ids": ["R__", "R__"],
    "overall_sentiment": "positive | mixed | negative | unclear"
  }
}
```

## Quality Rules for Structured Output

| Rule | What it means |
|---|---|
| Valid JSON | The JSON should parse without errors. Do not include Markdown fences around the final JSON unless your instructor asks for them. |
| Evidence-based extraction | Every issue category, urgency label, and recommended action should be supported by the raw note. |
| No hallucination | Do not add explanations such as server problems, GPS failure, or policy reasons unless explicitly stated. |
| Use null correctly | If a date, action, or missing detail is not in the record, use null or list it under missing_information. |
| Consistent labels | Use the same label for the same type of issue across records. |
| Concise summary | The Markdown report should be easy for a manager to scan in less than two minutes. |

## What to Submit

- Your summarization prompt and your extraction prompt.
- The final Markdown summary report.
- The final JSON extraction output.
- A short validation note: state whether the JSON is valid and list one correction you made.

- A 150–200 word reflection.

## Optional Extension Challenge

- Run the task once with a weak prompt and once with your improved structured prompt. Compare the outputs in 5–7 sentences. Focus on accuracy, consistency, missing values, and readability.
- Create a second JSON schema for a different stakeholder, such as the app engineering team or the campus safety team.

## Grading Rubric

| Criterion | Points | Excellent work shows… |
|---|---|---|
| Prompt design | 20 | Prompt includes role, task, context, format, constraints, evidence requirement, and no-hallucination |
| Markdown summary | 20 | Summary is concise, well-structured, uses record IDs, identifies themes, risks, positive feedback, and next steps. |
| JSON extraction | 30 | JSON is valid, complete for all 12 records, uses controlled labels consistently, and handles missing information correctly. |
| Validation and revision | 15 | Student checks parseability, consistency, evidence support, and documents at least one improvement. |
| Reflection | 15 | Reflection clearly explains what structured templates improved and identifies one limitation or failure mode. |

## Instructor Notes

- This exercise works well after students have seen examples of role prompting, task decomposition, and output templates.
- For beginners, let students complete the Markdown summary first, then the JSON extraction.
- During discussion, compare outputs across students and ask: Which fields were ambiguous? Which labels were overused? Which prompt rule prevented hallucination?
- A common teachable mistake is allowing the model to add technical causes not stated in the raw notes. Use this to reinforce evidence-based prompting.